Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

RouteLens: Traffic-Aware Multiple Routes Visualization and Optimization System

Team Name: *Team #*	*Carpe Diem* *DSCPP-III-2026-T278*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Upreti, Gunjita – 2510382973* *2510382973@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Rawat, Sneha – 2510012141* *2510012141@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rawat, Aayush – 2510370528* *2510370528@geu.ac.in* 

PHASE-III PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

RouteLens is a web-based route planner for Dehradun. Rather than handing commuters a single "fastest" route, it lets them compare several and pick the one that suits them. Given a start point and a destination, it finds up to three genuinely different routes on the real OpenStreetMap road network (31,802 nodes, 72,626 directed edges) and draws each in its own colour, with its ETA, distance, number of major junctions and a Low/Medium/High traffic level. Users say how much they care about time, distance, traffic and junctions, and RouteLens turns this into one weighted score to recommend a route. What-if scenarios (rain, rush hour, an incident or a closed road) show how the options change.

Underneath, the system runs on core data structures and algorithms: an adjacency-list graph, Dijkstra's algorithm with a binary min-heap, the penalty method for alternative routes, BFS and a hash table. In Phase III we are adding offline traffic data: TomTom speeds will be fetched before the demo and saved to a file, so the demo does not need internet.

Updated Project Approach and Architecture (2 pts)

1. Data preparation (Python). One script exports the Dehradun road network from OpenStreetMap into CSV files. A second script will pull TomTom traffic speeds for the main roads before the demo and store them in `traffic.csv`.

2. Routing engine (C++). The loader builds the road graph as an adjacency list. A hash table maps OSM IDs to node numbers, and only the connected part of the network is kept. Roads without traffic data fall back to their normal (free-flow) speed; the reader for `traffic.csv` is being added now. Each road gets a single cost based on the user's priorities (time, distance, traffic, junctions). Dijkstra's algorithm with a binary min-heap finds the best route. For the 2nd and 3rd routes, the penalty method makes roads that are already used more expensive and runs Dijkstra again, and any route that is almost the same as an earlier one is dropped. Scenarios (rain, rush hour, incident) change road speeds, and BFS checks that the destination can still be reached.

3. Server and website. A Crow HTTP server sends routes to the website as JSON (`/api/routes`, `/api/scenario`). The website draws them with Leaflet on OpenStreetMap tiles, and will also show when the traffic data was collected.

Tools: C++ with STL, Crow, Python, Leaflet, CMake, GitHub Actions, Docker.

Tasks Completed (7 pts)

Task Completed	Team Member
Dijkstra with binary min-heap, penalty-method alternative routes, weighted route scoring, BFS reachability for road closures; correctness check against NetworkX (PR #16).	Upreti, Gunjita (Team Lead)
OSM export and loader with hash table (OSM ID to index); ETA with free-flow fallback; traffic index and Low/Medium/High levels; scenarios (rain, rush hour, incident); 41 documented major junctions (PRs #6, #12–#15).	Rawat, Sneha
Crow server with `/api/routes` and `/api/scenario` connected to the routing engine; Leaflet website (three coloured routes, info boxes, priorities, what-if closure UI); major junctions in the server; CI and Docker (PRs #1, #3–#5, #7–#11, #17).	Rawat, Aayush

Challenges/Roadblocks (7 pts)

1. Keeping the design simple. Some of the methods we tried turned out to be more complex than this project needed. *Solution:* we stuck with standard methods (Dijkstra, the penalty method, BFS, a hash table). Users get the same results, and the code is easier to test and explain.

2. The demo depends on internet and the TomTom API limit. *Solution:* we will fetch traffic before the demo and save it to a file, so the demo can run offline. The website will show when the data was fetched.

3. OpenStreetMap has too few traffic signals for Dehradun. *Solution:* we use our own documented list of 41 major junctions in the city.

4. Open question. We still need to check with the evaluators that C++ with STL is acceptable for the data structures.

Project Outcome/Deliverables (2 pts)

• A working RouteLens website, backed by a C++ routing engine that runs on the real Dehradun road network.
• Up to three different routes for each query, each with its ETA, distance, junction count and traffic level, plus one recommended route.
• What-if scenarios for rain, rush hour, an incident or a road closure.
• TomTom traffic data saved ahead of the demo, so it can run offline.
• Python scripts that prepare the road and traffic data.
• Unit tests, GitHub Actions CI, a Docker image and project documentation.

Progress Overview (2 pts)

Done: the road data and loader; routing (Dijkstra with a min-heap, the penalty method, BFS and the weighted score); the traffic model (ETA, traffic levels, scenarios, junctions); the server; the website with what-if mode; and CI and Docker. The whole flow already works end to end: pick two points, see up to three routes with their ETA, traffic level and junctions, and try out scenarios.

Ahead of schedule: we have already checked the routing results against NetworkX.

Still to do: save traffic data to a file before the demo and read it in C++, show the time of that data on the website, tidy up parts of the routing code, and run usability tests with classmates.

Codebase Information (2 pts)

Repository: https://github.com/gunjita18/RouteLens-DSCPP
Branch: `main` (member branches `feature/routing-algorithms`, `sneha`, `aayush`, merged through pull requests)

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit tests (20 CTest targets: heap, Dijkstra, penalty, closures, scoring, loader, traffic, scenarios, junctions, server)	Pass	Run on GitHub Actions CI, 1 Nov 2026. Snapper checks on the real network are skipped in CI because the CSVs aren't available there.
Correctness against NetworkX	Pass	1,000 pairs $\times$ 3 profiles with no mismatches
Alternative routes (100 pairs)	Pass	Three different routes found for 96% of pairs
Docker build and `/api/health` smoke test	Pass	Run in CI
Traffic file loading tests	Pending	Will be added along with the traffic file
Usability test with classmates	Pending	Will include checks on a phone and for colour-blind users

Deliverables Progress (2 pts)

- Routing engine (Dijkstra, penalty method, BFS, weighted score): **Completed**.
- Road network and loader: **Completed**.
- Traffic model (ETA, levels, scenarios, junctions): **Completed**.
- Traffic data saved before the demo (Python script and C++ reader): **Pending**.
- Server and website with what-if mode: **Completed**; showing traffic data time: **Pending**.
- Tests, CI and Docker: **In progress**.
- Documentation: **In progress**; usability testing: **Pending**.